

# altairsim — The Monitor

---

2026-08-22 · 1c21644

*A simulation of the MITS Altair 8800 and the S-100 bus*

|                                                   |   |
|---------------------------------------------------|---|
| The monitor .....                                 | 1 |
| Commands resolve by prefix .....                  | 1 |
| Editing the command line .....                    | 2 |
| Repeating the last command: . .....               | 3 |
| Reaching the host: ! .....                        | 4 |
| Numbers: one rule, and it is not negotiable ..... | 4 |
| Naming a board: <id>[:<unit>] .....               | 5 |
| Seeing the machine .....                          | 6 |
| Changing the machine .....                        | 7 |
| Running, and stopping .....                       | 7 |
| Speed .....                                       | 8 |
| RESET is not POWER .....                          | 8 |

# The monitor

---

This is one of two short documents about **driving `altairsim` itself** — the program you run, as opposed to the machines it emulates. It covers **the monitor**: the `altairsim>` prompt where you start, stop, examine and reconfigure the machine. Its companion, *The Debugger*, covers what you reach for at that prompt when something has gone wrong. Both stand beside the *User Manual*, which describes the hardware — the boards, the machines, the disks and the tapes. `altairsim` simulates the MITS Altair 8800 and the S-100 bus it was built around; if that is new to you, the manual's opening chapter is the place to start.

The `altairsim>` prompt is **the monitor**: the front panel of the machine, and its debugger, which here are the same thing. Everything the front panel of a real Altair could do, the monitor can do — and a great deal it could not. It breakpoints, it single-steps, it disassembles, and it will show you the bus itself: who decodes what, who is pulling which interrupt line, and where two boards are fighting over an address. That is what *The Debugger* is for, and it is most of why this program exists.

What it is not is a *menu* — a layer sitting between you and the machine, offering a fixed set of things it is prepared to let you inspect. There is no debug mode to enter and nothing is watching from the outside. A breakpoint is **the machine stopping**, not a script noticing that it should have. `IN` and `OUT` run **real bus cycles**, with every side effect a real one has — read a UART's data port at this prompt and you have taken the byte, exactly as the guest would have. The panel and the debugger are one object because on an Altair they were one object: a man at the switches, reading lamps.

The machine does not have to be running for the monitor to work. Most of what follows — examining memory, running a bus cycle, fitting a board — works on a machine with the power on and the processor idle, which is exactly the arrangement the front panel was for.

## Commands resolve by prefix

There are **no aliases and no memorised abbreviations**. You type as much of a command as it takes to be unambiguous, and the first command that matches wins.

`HELP` prints the whole menu with each command's shortest form in brackets:

|              |            |           |           |
|--------------|------------|-----------|-----------|
| BO[ARDS]     | B[REAK]    | COM[PARE] | C[ONFIG]  |
| CONN[ECT]    | CONS[OLE]  | DE[POSIT] | DI[SASM]  |
| DISC[ONNECT] | D[UMP]     | E[DIT]    | EX[AMINE] |
| F[ILL]       | HE[LP]     | H[ISTORY] | I[N]      |
| L[OAD]       | M[OUNT]    | MOV[E]    | N[EXT]    |
| NO[BREAK]    | O[UT]      | P[OWER]   | Q[UIT]    |
| REGI[ON]     | RE[GS]     | RES[ET]   | REST[ORE] |
| R[UN]        | SA[VE]     | SEA[RCH]  | SE[T]     |
| SH[OW]       | SN[APSHOT] | S[TEP]    | SY[MBOLS] |
| T[RACE]      | TY[PE]     | U[NMOUNT] | W[H0]     |

Type the part before the bracket. `D` is `DUMP`; `DE` is `DEPOSIT`; `RES` is `RESET`. Case does not matter, here or in the name of any board.

`HELP <command>` gives you the usage and worked examples for one of them, and `?` is the same as `HELP`.

**R is RUN, not RESET.** It is the command you type every session, and it is the one that costs nothing if you did not mean it. A bare `R` that reset the machine would be one you had to set up again. `RESET` pays the letters: `RES`.

## Editing the command line

The prompt is a real line editor. The key labelled Backspace erases the character behind the cursor whichever byte your terminal sends for it, the arrows move within the line, and the editor keeps a **command history** — the up-arrow walks back through the lines you have typed and the down-arrow returns toward the one you were in the middle of.

The history is **saved between sessions, per directory**. When you leave, the last commands you typed are written to a hidden `.altairsim_history` in the directory you launched from, so the next time you start the simulator *there* they are waiting on the up-arrow. Each project directory keeps its own list; the file is written only when you are typing at a real terminal, so a script, a pipe, or an automated run never leaves one behind. How many lines it keeps is the `history` setting on the console — `SET CONSOLE history=200` to keep more, and `SET CONSOLE history=0` to turn the file off entirely. It defaults to 50.

## Completing with `Tab`

`Tab` finishes whatever you are partway through typing, and it reads the candidates off the machine in front of you — so a board you plug in is completable straight away, with nothing to keep up to date:

- at the start of a line, a **command** — `SH` → `SHOW` ;
- after `SET` , a **board id** (or `CONSOLE` , `DISPLAY` ) — `SET me` → `SET mem0` ;
- after a board, one of **its property names**, with the `=` put on ready for the value — `SET mem0 fi` → `SET mem0 fill=` ;
- after the `=` , one of that property's **legal values** — `SET mem0 fill=` → offers `zero` and `random` .

When more than one candidate fits, `Tab` fills in as far as they all agree and stops; press it again and it lists them. When nothing fits, it does nothing.

## The keys

| Key                                      | Does                                                    |
|------------------------------------------|---------------------------------------------------------|
| <code>←</code> <code>→</code>            | move one character                                      |
| <code>Ctrl-A</code> / <code>Home</code>  | to the start of the line                                |
| <code>Ctrl-E</code> / <code>End</code>   | to the end of the line                                  |
| <code>Alt-B</code> / <code>Ctrl-←</code> | back one word                                           |
| <code>Alt-F</code> / <code>Ctrl-→</code> | forward one word                                        |
| <code>Backspace</code>                   | erase the character before the cursor                   |
| <code>Delete</code>                      | erase the character under the cursor                    |
| <code>Ctrl-W</code>                      | erase the word before the cursor                        |
| <code>Ctrl-K</code>                      | erase from the cursor to the end of the line            |
| <code>Ctrl-U</code>                      | erase the whole line                                    |
| <code>↑</code> <code>↓</code>            | walk back and forth through the command history         |
| <code>Tab</code>                         | complete the word at the cursor                         |
| <code>Ctrl-D</code>                      | on an empty line, leave — the same as <code>QUIT</code> |

**Ctrl-E here is end-of-line, not ATTN.** At the prompt you are typing to the *editor*, so `Ctrl-E` jumps to the end of the line. Once a **running** guest holds the console, that same `Ctrl-E` is **ATTN** and takes the keyboard back (below). Same key, two places, two jobs.

## Repeating the last command: `.`

Type `.` on a line by itself and the monitor runs your **last command again**, quietly — there is no echo, just the command's own output. It costs one keystroke, and the commands you most often want to repeat

pick up where they left off: a bare `DISASM` disassembles the next screenful, a bare `DUMP` shows the next page, and `STEP` steps again. So you type `DI` once and then `. . .` to walk forward through a routine, or `S` and then `.` to single-step.

Pressing `.` again always repeats that same original command, never the previous `.`, so it keeps doing the one thing however many times you press it. A `.` before you have typed anything just tells you there is nothing to repeat yet.

## Reaching the host: `!`

A line that begins with `!` is not a monitor command at all — everything after the `!` is handed to **your host shell**, word for word, and the monitor waits until it is done before it prompts again. The rest of the line is passed through untouched, spaces and all:

```
!ls                list the directory you started from
!vi HELLO.PRN      open a file in your editor, then :q back to the prompt
!cp game.dsk save.dsk  keep a copy of a disk without unmounting it
```

This is **your** shell, with your own privileges — not the machine's, and nothing the guest can see or reach. It is also why an editor works: the monitor is not holding the keyboard when it hands off, so `vi` gets a normal terminal and gives it back when it exits. The machine keeps running underneath; `!` only borrows *you*, not the processor.

A bare `!` with nothing after it just reminds you of the form.

## Numbers: one rule, and it is not negotiable

**On the wire → hex. Never on the wire → decimal.**

If the 8080 can see it, it is **hex**: an address, a port, a data byte, a register. If it never leaves your head, it is **decimal**: a count, a width, a size, a drive number.

**Hex is only the *default* for the wire class.** Switch the console to octal and that class reads and prints in octal instead — the base the MITS manuals and the front panel spoke. The rule does not change: octal is still the wire class, decimal is still the counts. *Reading and writing in octal*, below, is how.

```
DUMP 100           address  -> 0100 hex
STEP 10            a count  -> ten instructions
OUT FF 55          port and byte -> both hex
SET sio0:a baud=9600  a baud rate -> nine thousand six hundred
```

You can always force the issue: `0x`, `$` and a trailing `h` force hex; `0o` and a trailing `q` force octal; `0b` forces binary — which is what you want for the front panel's sense switches, where eight switches would rather be eight digits; a leading `#` forces decimal; and a `K` or `M` suffix is **always** decimal ( `48K` is 49,152 — so `0x10K` is a contradiction and is rejected rather than guessed at).

This rule is the same everywhere — in the monitor, in a machine file, and in every board's settings. There is no second convention to learn.

What is not negotiable is the **classes** — which side of the line a number falls on. The base the wire class is *printed* in is yours, and the next section is how.

## Reading and writing in octal

The MITS manuals and the Altair front panel spoke **octal**, not hex, and you can too:

```
SET CONSOLE base=octal
```

Now the **hex** half of the rule becomes **octal** — the wire class (addresses, ports, data bytes, registers) is read and printed in **split octal**, each byte its own `000 – 377` group and a 16-bit address as two of them, exactly the way the front-panel address lamps are grouped:

```
EXAMINE 100      -> 000 100 076   (the byte 0x3E at address 0x40)
DUMP 100-100     -> 000 100 076
DISASM 0         -> JMP 022 064    (a jump to 0x1234)
```

A bare number is octal now too, so `100` is address `0x40`; the decimal class (counts, widths, baud) does not change. The forcing markers still work in both directions — `0x1234` is hex even in octal mode, and `0o377` is octal even in hex mode — so nothing is ever a base you cannot type your way out of. `base=hex` (the default) puts it back. Set it once in a machine file ( `[console] base = octal` ) to start there every time.

## Naming a board: `<id>[:<unit>]`

Every board in the machine has an **id** you chose ( `cpu0` , `sio0` , `dsk0` ), and some boards have **units** inside them — the two channels of a serial board, the four drives on a floppy controller, the ROM socket on a memory board.

```
SHOW sio0          the board
SET sio0:a baud=1200 one channel of it
MOUNT dsk0:drive1 my.dsk
```

You may leave out anything that carries no information. If there is only one floppy controller in the machine, `dsk` will find it. If a board has only one thing you could mount into, you need not name it — `MOUNT ACR tape.bin` puts a cassette in the one recorder.

But anything genuinely plural you must say. There are four drives on that controller and the machine will not guess which one you meant; it will tell you so and stop.

## Seeing the machine

```
altairsim> BOARDS
  ID   TYPE      I/O      UNITS      MEMORY
  ---
fp0   fp         FF        -           -
cpu0  8080         -        1 cpu: 8080 -
sio0  2sio        10,12     2 serial: a*, b -
dsk0  dcdd        08,09,0A  4 disk: drive0(empty), ... -
hb0   hostbridge  B0,B1     -           -
mem0  memory        -        1 rom: rom0  0000-DFFF ram 56K
                                FF00-FFFF rom dbl phantom:all

* holds the console
```

That is the backplane: what is plugged in, what ports each board answers to, what is in its units, and what it decodes in memory.

| Command             | Shows                                                                                            |
|---------------------|--------------------------------------------------------------------------------------------------|
| BOARDS              | the backplane                                                                                    |
| SHOW <id>           | one board: every setting, its value, and what it will accept                                     |
| SHOW MACHINE        | the whole machine                                                                                |
| SHOW CONSOLE        | which unit holds your keyboard, and how bytes are being transformed                              |
| SHOW DISPLAY        | the video window: whether it or the terminal has the keyboard, and whether it wears the CRT look |
| SHOW JOYSTICKS      | the host game controllers a D+7A can read (needs an SDL3 build)                                  |
| SHOW BUS MAP        | who decodes which addresses — and what floats                                                    |
| SHOW BUS IO         | who decodes which ports                                                                          |
| SHOW BUS IRQ        | who is strapped to which interrupt line, and who is pulling it                                   |
| SHOW BUS CONTENTION | where two boards are fighting                                                                    |

`SHOW <id>` is worth dwelling on, because it is the **only** thing you need in order to configure a board. It lists every property, what it is set to, and what values are legal — and those property names **are** the keys you write in a machine file. There is no second schema anywhere in this program. The board reference at the back of this manual is printed from the same source.

## Changing the machine

```
SET cpu0 clock_hz=2000000    give it the real 2 MHz crystal
SET mem0 fill=zero           RAM comes up zeroed instead of random
SET fp0 sense=80             set the SENSE switches
BOARDS ADD 2sio sio1 port=20  fit a second serial board
BOARDS REMOVE sio1           pull it out
CONFIG SAVE mine.toml         write out the machine you are actually running
```

`CONFIG SAVE` round-trips: what it writes, `altairsim mine.toml` will boot.

## Running, and stopping

```
RUN FF00    load the PC and go — the same two motions as the panel's switches
RUN         carry on from wherever the processor is
```

`RUN <addr>` is **EXAMINE** followed by **RUN**, exactly as you would do it on the front panel. There is no `BOOT` command in this program, and there should not be: a machine that ought to start says so with the operator's own keystroke.

If a board holds the console, **the guest gets the keyboard** — every key, including `^C`, which a CP/M program is entitled to read.

## ATTN takes it back

`^E` is **ATTN**. The host intercepts it *before the guest is ever offered the byte*, so no program running inside the machine can disable it, trap it, or take it from you.

```
A>
ATTN -- the machine is still at CA9C. RUN resumes.
altairsim>
```

**ATTN stops the machine and gives you the monitor.** Nothing executes while this prompt is up. But it stops the machine without *disturbing* it — **ATTN** is not **RESET** and not **POWER**, so the registers, the

memory and the disk are exactly as the guest left them, and a bare `RUN` (no address) picks up at the very instruction it was about to execute. That is what "still at CA9C" is telling you.

`CONSOLE attn=1D` moves `ATTN` to `^J` if `^E` collides with something the guest wants.

## What stops it for real

A `RUN` ends when it hits a **breakpoint**, or a `HLT` that nothing can wake — and it always says which. With no console connected there is nothing to hand the keyboard to, so it simply runs, and `^C` stops it.

**RUN and ATTN are the panel's RUN and STOP.** `RUN` starts the processor; `ATTN` — or a breakpoint, or a `HLT` — stops it and hands the monitor back. That prompt is the whole distinction: **the monitor exists only while the machine is stopped**. While it runs, the guest holds the keyboard and there is no `altairsim>` to type at; when you have the prompt, nothing is executing. So every `SET`, `DEPOSIT` and `EXAMINE` you type acts on a *stopped* machine — a property is never changed out from under a running instruction, and none is ever locked "while running" or settable only then.

## Speed

**It runs flat out by default** — `clock_hz` on the CPU board is `0`, so a cassette that took a real Altair 110 seconds comes off in about one. `SET cpu0 clock_hz=2000000` buys back the 2 MHz machine; what the guest sees is identical either way, because the tape still costs the same T-states — the crystal buys period *feel*, not *behaviour*. `SHOW cpu0` reports `achieved_hz` beside it: the clock the run loop actually hit, a measurement you cannot set.

The one exception is anything the guest times against the *outside* world — an XMODEM transfer wants the real crystal, a cassette does not. The manual's Boards chapter ( `clock_hz` , `idle` ) and its Troubleshooting chapter have the detail.

## RESET is not POWER

|       |                                                                                                                   |
|-------|-------------------------------------------------------------------------------------------------------------------|
| RESET | the bus's RESET* line. The processor restarts at <code>0000</code> . <b>Memory survives</b> , disks stay mounted. |
| POWER | a power cycle. <b>This is the only thing that loses RAM</b> and re-reads the ROM images.                          |

`RESET` does not clear memory because pressing RESET on a real Altair did not clear memory — that is behaviour a lot of period software depends on.

`RESET*` is a **line on the backplane**, not an instruction the simulator carries out for you, and every board hears it and answers the way its own silicon did — which is not the same answer twice. The memory board clears its bank latch but touches no RAM (a RAM chip has no reset pin); the floppy controller flushes the sector it was writing and deselects the drive; and the 2SIO does **nothing at all**, because the

6850 has no reset pin for `RESET*` to land on — so its baud rate, word format and interrupt enables all survive a reset, exactly as on the bench. Hit `RESET` mid-write and you get what the hardware gave you: a half-written sector, a serial port still configured as the dead program left it, and every byte of RAM intact.

**POWER is a different wire.** Switching the machine on drives `POC*` — Power-On Clear, its own backplane line — and a board may treat the two differently, because the real cards did. The 88-VI/RTC is the case that proves it: POC disables the board and `RESET*` is not wired to it at all, so an interrupt controller a crashed program left armed **stays armed through a `RESET`** and only clears on `POWER`. `POC*` is also the only moment RAM is allowed to forget: on `POWER` the memory board refills itself — with **random bytes by default**, because static RAM does not come up zeroed — and re-reads every ROM image.

|       | The processor                 | The boards                                                                              | RAM                            |
|-------|-------------------------------|-----------------------------------------------------------------------------------------|--------------------------------|
| RESET | restarts at <code>0000</code> | <code>RESET*</code> on the bus; each board answers as its silicon did — some do nothing | <b>survives</b>                |
| POWER | restarts at <code>0000</code> | <code>POC*</code> on the bus; the boards come up as they do from cold                   | <b>refilled</b> , ROMs re-read |